

Report 2: Project Progress

University/Department: California State University, Dominguez Hills

Course/Thesis/Project Title: CSC 599 Project

Semester/Year: Fall 2025

Project Title: A Hyper-Personalized Financial Product Engine Using
Generative AI

Student Name: Sumaiya Shaikh

Student ID: 213028116

Instructor / Committee Chair: Dr. Benyamin Ahmadnia

Date: 11-09-2025

1. Abstract / Summary of Progress

This report marks the completion of the "Foundation & Data" workstream (Weeks 1-2) and details the **critical analysis and design phase** for the "Model Building & Integration" workstream (Weeks 3-6) of the project. A supplementary literature review was conducted to analyze existing solutions and identify key research gaps. This analysis has validated the project's core objectives and provided a strong theoretical framework for the features planned for Weeks 7-9, specifically "Smart Spending Analysis" and "Proactive Alerts."

This document provides a detailed rationale for selecting the project's core machine learning architecture: a **Hybrid Recommender System** [1] and a **Retrieval-Augmented Generation (RAG) Pipeline** [2]. The "Work Completed" section (Section 3) details this new analysis. The "Proposed System Architecture" section (Section 4) is significantly expanded to describe the logical design, data flows, and evaluation strategies for these models in sufficient detail. The project is on schedule and is now prepared to move from this in-depth design phase to the implementation phase.

2. Introduction and Objectives (Recap)

This project seeks to address a key problem in the modern FinTech environment. In this section we present a summary of the project's problem, motivation, goals, and key ideas as they relate to the problem area.

2.1. Problem Statement

The primary problem being certainly a breakdown in ability to not just communicate but also fulfill matching individuals with highly relevant and contextual financial products in a safe digital environment. The digitalization of banking has not clarified choices; in fact, it has confused consumers and resulted in "decision paralysis" which dramatically decreases engagement [4]. Current environments are fragmented and contain a fundamental issue that exists:

- **Prediction without Explanation:** Traditional recommendation systems claim to be proficient in predicting the product a user well like (i.e., collaborative filtering), but do not provide an explanation that is personalized, relevant, and trustworthy in relation to the individual.
- **Explanation without Context:** In contrast, a modern "AI chatbot" may be able to explain complex financial concepts, but lacks deep personalized contextual detail. It is simply not aware of a user's transaction history, financial goals, nor their risk appetites as historical data, and subsequently produces generic, impersonal advice.

As a result, there is low customer engagement, low financial literacy and a significant loss of opportunity for both consumers as individuals and financial institutions.

2.2. Motivation and Scope

The project goals motivate the development of a single unified structure to fill the gap of a "trustworthy financial assistant" that securely ingests and utilizes granular financial data to give back to users. This system will merge prediction (the Recommender System) with explanation (Generative AI) into one process that users can access in a conversational manner.

- **Scope:** The scope of the project is to deliver a complete, end-to-end, technical proof- of-concept. This proof-of-concept will involve a secure user authentication and management system, a backend data architecture, a working predictive recommender model, and a conversational generative AI interface. The deliverable will be a web application, which is a full stack application that demonstrates this functionality of integration.
- **Limitations:** The recommender model will be trained only with the synthetic financial transaction data generated for this project. The generative AI will leverage the product catalog and user recommendations but will not be producing real-time market advice or executing trades.

2.3. Project Objectives

There are six primary objectives for the project established in Report 0:

1. **System Design:** Plan an end-to-end system topology for the web-based system (Completed).
2. **Data Synthesis and Analysis:** Create a realistic synthetic dataset of user financial behavior and products (Completed during this Period).
3. **Model Building:** Construct and train a machine-learning recommender model to recommend products by matching user profiles (Next Phase).
4. **AI Application:** Create an Generative AI model (RAG) to support conversational explanations and financial planning applications (Planned).
5. **Interface Development:** Toward building a complete fully functioning full-stack web application for users interaction (Planned).
6. **Evaluation:** Evaluate the overall system performance by reviewing model metrics, conversation quality, and systematic reviews (Planned).

3. Work Completed (Analysis & Design Phase)

Following the completion of the foundational work (Report 1), this reporting period (Weeks 3-6) has focused on the critical analysis and design for the two core AI modules.

3.1. Research & Refinement of Project Scope (Detailed Analysis)

To ensure the project's features are high impact, a supplementary literature review was conducted (via Google Scholar, IEEE Xplore, and ACM Digital Library) to validate the innovative potential of this project.

- Gap 1: Reactive vs. Proactive: The vast majority of current tools are reactive; they show users where their money has gone. The academic literature shows a clear and unmet need for proactive tools that can accurately forecast personal cash flow and alert users before a financial event [3].
 - Analysis: This finding is critical. It implies that simply showing a user a pie chart of their spending (a common feature) has diminishing returns. The real value is in shifting the user's perspective from past to future. This strongly supports the "Proactive Alerts" feature, reframing it as the central value proposition of the dashboard. Our ML model, therefore, must be a forecasting model, not just a categorization one.
- Gap 2: The "Youth Chasm": A significant finding is the lack of effective digital financial tools for users in the 14-20 age bracket. Research shows that financial literacy interventions are most effective when targeted at adolescents, yet this group is largely underserved by current FinTech products [4].
 - Analysis: This gap presents both a market opportunity and a clear path to demonstrating "hyper-personalization." A 16-year-old's financial "problem" is not portfolio management; it's understanding compound interest and managing allowance. Our system's "Life-Stage" feature will be more than a cosmetic change; it will load a completely different, simpler set of rules and goals for this demographic.
- Gap 3: Superficial Behavioral Nudges: Many apps use simple, generic "nudges" (e.g., "You've spent \$50 on coffee!"). Research in behavioral economics confirms that for nudges to be effective, they must be timely, relevant, and personalized [5].
 - Analysis: The term "nudge" is often misused. A generic alert is just data. A true nudge leverages "choice architecture" [5] to make a better decision easier. Our analysis concludes that our RAG pipeline is the key to this. Instead of an alert, the chatbot can proactively ask, "You're near a coffee shop where you've spent \$80 this month. Would you like to move \$5 into your 'Vacation' goal instead?" This is a specific, actionable, and timely intervention that our system is uniquely designed to provide.

3.2. Analysis and Selection: Recommender Module

Based on the project goals and the nature of the synthetic dataset, a Hybrid Recommender System [1] has been selected as the optimal approach. A detailed architectural design is presented in Section 4.

- Reasoning: The system must be able to handle the "cold start" problem, where new users have no transaction history. A single model (like Collaborative Filtering) would fail for these users.
- Selected Solution: A hybrid model that combines a Content-Based Filter for new users and a Collaborative Filtering (Matrix Factorization) model [6] for established users.

3.3. Analysis and Selection: Generative AI (RAG) Pipeline

To solve the "Prediction without Explanation" problem, a Retrieval-Augmented Generation (RAG) pipeline [2] has been designed.

- Reasoning: A standard Large Language Model (LLM) is not aware of the user's private data, the product catalogue, or the recommender's reasoning. This makes it unsafe for financial advice, as it can "hallucinate" (fabricate) information.
- Selected Solution: The RAG model solves this by grounding the LLM in facts. When a user asks a question, the system first retrieves all relevant, factual data and augments the LLM's prompt with this context. This ensures all explanations are personalized, accurate, and safe.

4. Proposed System Architecture & Model Design (Detailed)

This section provides the detailed technical design for the implementation phase. (A diagram, such as the one generated in `Project_Architecture_Diagram.html`, will be inserted here as Figure 1).

4.1. Proposed Recommender Logic Flow (Detailed)

This section provides the detailed technical design for the implementation phase.

4.1.1. Data Preprocessing & Matrix Generation

1. Source Data: Transaction table (user_id, product_id, timestamp).
2. Task: Convert this "long" list of interactions into a "wide" user-item interaction matrix.
3. Process:
 - A Python script will `pandas.read_sql()` the Transaction table.

- We will aggregate interactions. For this prototype, we will use a binary "interaction" (1 if the user has a product, 0 if not). A more advanced model might use transaction frequency or amount as the value.
 - The `pandas.pivot_table()` function will be used to create the final matrix: $M[user, item] = 1$ or 0 .
4. Sparsity: This matrix will be extremely sparse (mostly zeros), as users will only have interacted with a tiny fraction of all available products. This is the primary reason for our model selection.

4.1.2. Model 1: Content-Based Filter (Cold Start)

- Trigger: A user is considered "cold start" if they have < 5 non-zero entries in the interaction matrix.
- Logic: This model will execute a direct query against the User and ProductCatalog tables. It is a rules-based engine.
 - Step 1: Fetch user's "content" features (e.g., `user.risk_tolerance = 'low'`, `user.income_level = 'medium'`).
 - Step 2: Define a business logic mapping. For example:
 - If `risk_tolerance == 'low'`, recommend `product.category` in `['high_yield_savings', 'cd', 'debt_payment_plan']`.
 - If `risk_tolerance == 'high'`, recommend `product.category` in `['etf', 'index_fund', 'investment_loan']`.
 - Step 3: The system will return products that satisfy this filter, excluding any the user already has.

This approach is simple, fast, and provides explainable (if basic) recommendations for new users.

4.1.3. Model 2: Collaborative Filtering (Matrix Factorization)

- Trigger: User has ≥ 5 interactions.
- Technique: We will use Matrix Factorization (MF), specifically Truncated SVD (Singular Value Decomposition) from the scikit-learn library [6].
- Why SVD?
 - Traditional Collaborative Filtering (user-user, item-item) fails with very sparse data.
 - MF techniques, like SVD, are ideal for sparse matrices. They address sparsity by decomposing the large user-item matrix M into two smaller, "dense" latent factor matrices: U (user factors) and V (item factors).
 - U represents each user's "tastes" as a vector (e.g., `[propensity for risk, interest in long-term goals]`).

- V represents each item's "properties" as a vector (e.g., [risk-level, suitable-for-long-term]).
 - By multiplying U and V back together, we get a full matrix M' of predicted ratings, effectively filling in all the zeros.
- Implementation:
 1. The sparse M matrix will be fed into the TruncatedSVD model for training (this will be done "offline" for the prototype).
 2. When a user u requests a recommendation, we take their vector U[u] and predict their rating for all items.
 3. We sort these predicted ratings and return the Top-N (e.g., Top 5) products that the user has not already interacted with.

4.1.4. Model Evaluation Plan

- Offline Evaluation: Before deploying, we will split our synthetic data (80% train, 20% test). We will train the SVD model on the 80% and evaluate its ability to "find" the hidden 20%.
- Metrics: We will use Precision@k and Recall@k (e.g., at k=5).
 - Precision@5: Of the 5 products we recommended, how many were "correct" (i.e., in the user's test set)?
 - Recall@5: Of all the "correct" products in the test set, how many did we successfully find in our top 5 recommendations?
- This evaluation is critical for the final report to prove the model's effectiveness.

4.2. Proposed RAG Pipeline Logic Flow (Detailed)

The RAG pipeline [2] will be the core of the /api/chat endpoint.

4.2.1. The "Retrieve" Step

1. User Question: "Why did you recommend this Tech ETF?"
2. Intent & Entity Recognition: The system must first identify the "entities" in the question: user_id = current_user.id and product_name = 'Tech ETF'.
3. Parallel Context Retrieval: The system will fetch three sources of context:
 - Context 1 (User): db.session.query(User).get(user_id) -> Fetches profile (e.g., risk_tolerance: 'high').
 - Context 2 (Product): db.session.query(ProductCatalog).filter_by(name='Tech ETF').first() -> Fetches description (e.g., 'This ETF tracks the NASDAQ...').
 - Context 3 (Recommendation): recommender_service.get_reason(user_id, product_id) -> Fetches the reason for the recommendation (e.g., reason: 'High SVD match score (0.92)').

4.2.2. The "Augment" Step (Prompt Engineering)

- This is the most critical step for safety. We will assemble a "mega-prompt" using a specific template.
- **System Prompt:** "You are a helpful financial assistant for 'FinPersonalize'. You must answer the user's question using only the context provided below. Do not use any outside knowledge. Do not fabricate information. Be friendly and concise. If the context is not sufficient to answer, say 'I do not have enough information to answer that.'"
- Context Block:
 --- CONTEXT START ---
 [User Profile: Risk Tolerance is 'high', Stated Goals are 'long-term growth']
 [Product Information: 'Tech ETF' is a 'high-risk' product that tracks the NASDAQ-100 index.]
 [Recommendation Reason: 'High SVD match score (0.92)']
 --- CONTEXT END ---
- User Question: "Why this Tech ETF?"

4.2.3. The "Generate" Step (Safety & PII)

- PII Scrubbing: Before the prompt is sent to the external LLM API (e.g., Hugging Face), a scrubber will remove PII. For example, user.name or user.email will never be placed in the context.
- API Call: The final, safe prompt is sent to the LLM.
- LLM Generates: The LLM, bound by the prompt, generates a safe, grounded answer: "Based on your 'high' risk tolerance and 'long-term growth' goals, we recommended the 'Tech ETF' because it is a 'high-risk' product that had a high match score (0.92) with your financial activities."

4.2.4. Futureproofing (Note for Report)

- For this prototype, retrieving from the SQL database is sufficient.
- For a Future work the system with thousands of products, this would be slow. The "Retrieve" step would be replaced with a Vector Database. The ProductCatalog descriptions would be converted into vector embeddings and stored. The user's question ("Why this ETF?") would also be "vectorized," and the system would perform a high-speed similarity search to find the most relevant context, which is a more scalable approach.

5. Next Steps

The project is on track. The next steps are focused on the **implementation** of these designs, aligning with Weeks 7-9.

- **Weeks 7-9 (November):**
 - **Task 1: Implement Recommender Service**
 - `reco_data.py`: Script to generate and save the user-item matrix.
 - `reco_model.py`: Train and pickle (save) the TruncatedSVD model.
 - `reco_service.py`: Create the RecommenderService class in Flask.
 - `routes.py`: Add `/api/recommend` endpoint that calls the service.
 - **Task 2: Implement RAG Pipeline**
 - `chat_service.py`: Build the RAG service class.
 - Implement context retrieval (User, Product, Reason).
 - Implement prompt-engineering template.
 - Implement PII scrubbing logic.
 - `routes.py`: Add `/api/chat` endpoint that calls the service.
 - **Task 3: Implement Financial Planning Features (MVP)**
 - *Backend*: Create logic for Smart spending/savings analysis (e.g., simple time-series forecast based on past 30 days).
 - *Backend*: Create DB models and API endpoints for a simple Goals feature (e.g., `POST /api/goals`).
 - *Frontend*: Build the React components to display the forecast and the goal-tracking UI.
 - **Task 4: Implement Proactive Alerts**
 - *Backend*: Create a simple module (e.g., `SubscriptionReviewer`) that scans transactions for recurring payments and returns them to the UI.
 - **Task 5: Documentation**
 - Begin drafting final project documentation and presentation slides.
- **Week 10 (Late Nov - Early Dec):**
 - Final end-to-end system integration and debugging.
 - Prepare demo video/screenshots.
 - Finalize presentation and rehearse.

5. References

- 1) R. Burke, "Hybrid Recommender Systems: Survey and Experiments," *User Modeling and User-Adapted Interaction*, vol. 12, pp. 331–370, 2002.
- 2) P. Lewis, E. Perez, A. Piktus, et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," *Advances in Neural Information Processing Systems* 33 (NeurIPS 2020), pp. 9459-9474, 2020.
- 3) A. S. Sethi, S. Arora, and N. P. Singh, "Machine learning based personal cash flow forecasting," *2021 International Conference on Computing, Communication, and Intelligent Systems (ICCCIS)*, 2021, pp. 312-316.
- 4) D. Fernandes, J. G. Lynch Jr., and R. G. Netemeyer, "Financial literacy, financial education, and downstream financial behaviors," *Management Science*, vol. 60, no. 8, pp. 1861-1883, 2014.
- 5) E. J. Johnson, S. B. Shu, B. G. C. Dellaert, et al., "Beyond nudges: Tools of a choice architecture," *Marketing Letters*, vol. 23, no. 2, pp. 487-504, 2012.
- 6) Y. Koren, R. Bell, and C. Volinsky, "Matrix Factorization Techniques for Recommender Systems," *Computer*, vol. 42, no. 8, pp. 30-37, 2009.